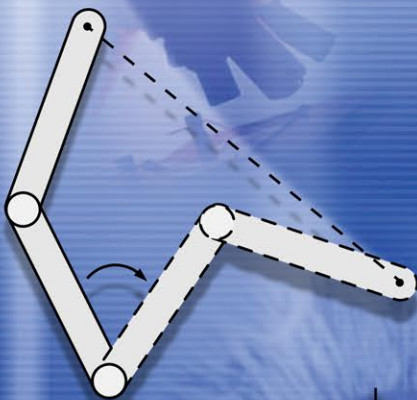


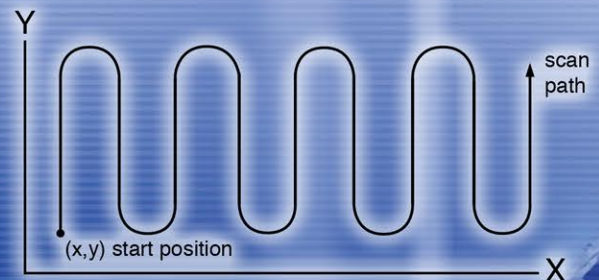
P M D PERFORMANCE**MOTION**DEVICES

MagellanMotionControl IC

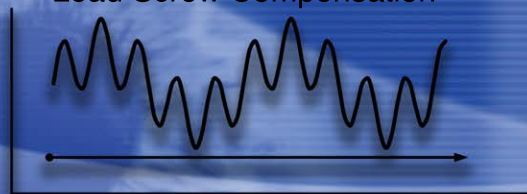
Inverse Kinematics



Multi-dimensional Contouring



Lead Screw Compensation



User-Defined Profile Mode

Custom Feature Documentation

Magellan™ Motion Control IC

User-Defined Profile Mode

Documentation



Performance Motion Devices, Inc.
80 Central Street
Boxborough, MA 01719



NOTICE

This document contains proprietary and confidential information of Performance Motion Devices, Inc., and is protected by federal copyright law. The contents of this document may not be disclosed to third parties, translated, copied, or duplicated in any form, in whole or in part, without the express written permission of PMD.

The information contained in this document is subject to change without notice. No part of this document may be reproduced or transmitted in any form, by any means, electronic or mechanical, for any purpose, without the express written permission of PMD.

Copyright 1998–2015 by Performance Motion Devices, Inc.

ATLAS, Magellan, ION, Magellan/ION, Pro-Motion, C-Motion, and VB-Motion are trademarks of Performance Motion Devices, Inc.

Warranty

PMD warrants that its products shall substantially comply with the specifications applicable at the time of sale, provided that this warranty does not extend to any use of any PMD product in an Unauthorized Application (as defined below). Except as specifically provided in this paragraph, each PMD product is provided “as is” and without warranty of any type, including without limitation implied warranties of merchantability and fitness for any particular purpose.

PMD reserves the right to modify its products, and to discontinue any product or service, without notice and advises customers to obtain the latest version of relevant information (including without limitation product specifications) before placing orders to verify the performance capabilities of the products being purchased. All products are sold subject to the terms and conditions of sale supplied at the time of order acknowledgment, including those pertaining to warranty, patent infringement and limitation of liability.

Unauthorized Applications

PMD products are not designed, approved or warranted for use in any application where failure of the PMD product could result in death, personal injury or significant property or environmental damage (each, an “Unauthorized Application”). By way of example and not limitation, a life support system, an aircraft control system and a motor vehicle control system would all be considered “Unauthorized Applications” and use of a PMD product in such a system would not be warranted or approved by PMD.

By using any PMD product in connection with an Unauthorized Application, the customer agrees to defend, indemnify and hold harmless PMD, its officers, directors, employees and agents, from and against any and all claims, losses, liabilities, damages, costs and expenses, including without limitation reasonable attorneys’ fees, (collectively, “Damages”) arising out of or relating to such use, including without limitation any Damages arising out of the failure of the PMD product to conform to specifications.

In order to minimize risks associated with the customer’s applications, adequate design and operating safeguards must be provided by the customer to minimize inherent procedural hazards.

Disclaimer

PMD assumes no liability for applications assistance or customer product design. PMD does not warrant or represent that any license, either express or implied, is granted under any patent right, copyright, mask work right, or other intellectual property right of PMD covering or relating to any combination, machine, or process in which such products or services might be or are used. PMD’s publication of information regarding any third party’s products or services does not constitute PMD’s approval, warranty or endorsement thereof.



Related Documents

Magellan Motion Control IC User's Guide

Complete description of the Magellan Motion Control IC features and functions with detailed theory of its operation.

Magellan Motion Control IC Programmer's Command Reference

Descriptions of all Magellan Motion Control IC commands, with coding syntax and examples, listed alphabetically for quick reference.

Table of Contents

1. User-Defined Profile Mode	9
1.1 Product and Hardware Configuration Summary	10
1.2 User-Defined Profile Mode Parameters.....	10
1.2.1 Settable Parameters	10
1.2.2 Readable Parameters	11
1.3 User-Defined Profile Mode Operation	12
1.3.1 User-Defined Profile Mode and Synchronization.....	13
1.3.2 Master Axis Source Selection.....	13
1.3.3 Adjusting the Incoming Data Stream via the Rate Scalar	15
1.3.4 Setting a Stop Value	16
1.3.5 Adjusting the Phase of the Incoming Datastream.....	16
1.3.6 User-Defined Contour Registers.....	17
1.4 Setting Up and Using User-Defined Contour Buffers.....	17
1.4.1 Shared I Buffer Arrays and Multi-Axis Synchronization	17
1.4.2 On-the-Fly Buffer Loading	18
1.4.3 Buffer Wrap Without On-the-Fly Data Loading.....	19
1.4.4 Buffer Wrap With Position Displacement	20
1.5 Controlling Contour Drawing.....	20
1.5.1 Contour Drawing Start Location	20
1.5.2 Contour Drawing Output Position	21
1.6 Entering and Exiting User-Defined Profile Mode.....	21



This page intentionally left blank.

List of Figures

1-1	Common Applications Using User- Defined Profile Mode	9
1-2	Dual-ported RAM and Standard RAM External Memory Buffer Configurations	10
1-3	Control Flow Overview of User-Defined Profile Mode	12
1-4	Simple Example User-Defined Contour	12
1-5	Example User-Defined Contour in a CAM Application	14
1-6	Master Axis S-Curve Profile	14
1-7	Master Axis Motor Connection Schemes	15
1-8	Example Two-Axis Synchronization Via Shared IBuffer	18
1-9	Example Position Output With Zero Displacement Wrap	19
1-10	Example Position Output With Non-Zero Displacement Wrap	20
1-11	Stored Two-Dimensional Contour Drawn in Three Separate Locations	21



This page intentionally left blank.

1. User-Defined Profile Mode

1

In This Chapter

- Product and Hardware Configuration Summary
- User-Defined Profile Parameters
- User-Defined Profile Mode Operation
- Setting Up and Using User-Defined Profile Buffers
- Controlling Where the Contour Is Drawn
- Entering and Exiting User-Defined Profile Mode

Some PMD Magellan IC, module, and card products are available with a capability for synchronized continuous path profile generation in one or more axes known as the user-defined profile mode.

Applications that may benefit from the user-defined profile mode include label cutting, 2-D and 3-D printing, glue application, metal and wood working, arbitrary path generation, and many others. In addition, this profile mode can be used to control kinematically complex mechanisms such as SCARA arms or other non-orthogonal linkages. Finally, this profile mode can provide a general purpose programmable cam capability.

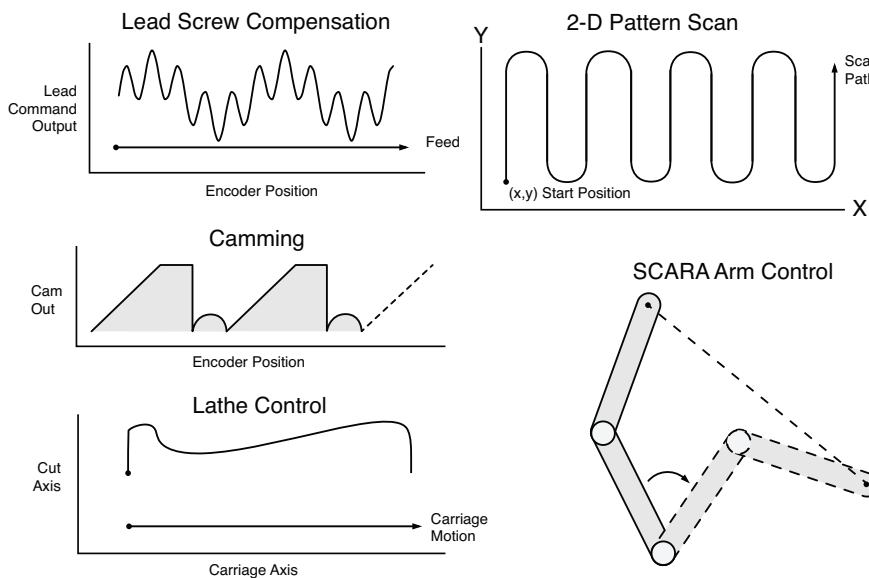


Figure 1-1:
Common
Applications
Using User-
Defined Profile
Mode

1.1 Product and Hardware Configuration Summary

The User-defined profile mode is not a standard Magellan IC feature but is available as a custom feature. Contact your PMD representative for information on how to order PMD products with this custom feature installed. All MC50000 Magellan Motion Controls ICs including the MC58113 Series of ICs support the user-defined profile mode, along with Magellan-based motion cards such as PMD's Prodigy series.

Unlike Magellan's other profiling modes, user-defined profile mode requires the availability of a memory buffer. See the Magellan Motion Control IC User's Guide for a general description of Magellan memory buffer architecture, setup, and access.

For Magellan MC50000 IC products except MC58113 Series use of a memory buffer means that external memory must be included in the user's card design. If a Prodigy motion card is used, external memory is already included. MC58113 Series Motion Control ICs support internal RAM buffers, and therefore do not require external RAM hardware.

[Figure 1-2](#) shows the Magellan IC connected to external memory via two memory read/write path options. Dual-port RAM provides a high speed path for loading in profile path information from the host. Single ported RAM is simpler to implement but means that loading and reloading of path information must occur through the Magellan.

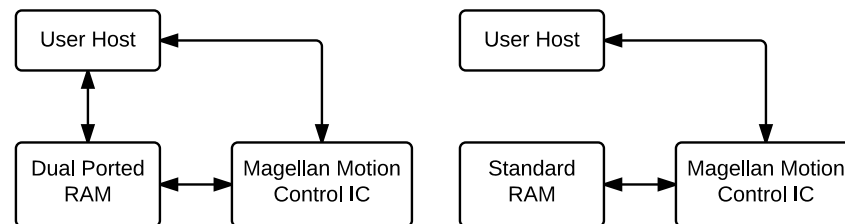


Figure 1-2:
Dual-port RAM and
Standard RAM
External
Memory Buffer
Configurations

1.2 User-Defined Profile Mode Parameters

1.2.1 Settable Parameters

The following table summarizes the host-settable parameters for the user-defined profile mode.

Profile Parameter	Format	Word Size	Range
Master Source	-	2 bits	3 values; encoder, commanded position, or time
Master Axis # -	8 bits	0-255	1-4 depending on Magellan type
Rate Scalar	16.16	32 bits	-32,768 to 32,767 + 65,535/65536
Start Value	32.0	32 bits	-2,147,483,648 to 2,147,483,647
Start Index	32.0	32 bits	0 to 2,147,483,647

Profile Parameter	Format	Word Size	Range
Stop Value	32.0	32 bits	-2,147,483,648 to 2,147,483,647
IBufferId	-	-	0-31
PBufferId	-	-	0-31

Below is a brief summary of these settable parameters. Each of these will be explained in more detail in the sections that follow.

Master source selects the source of the incoming position or time datastream used with the user-defined contour. This parameter can be set to one of three value; the actual encoder position of the master axis, the commanded output position of the profile generator of the master axis, or the Magellan's master time clock value which starts at zero at reset and increases thereafter.

Master axis number selects the axis number of the incoming position or time datastream used with the user-defined contour.

Rate scalar allows the incoming master source datastream to be slowed down, reversed, or sped up.

Start value and *start index* set the location within the overall specified contour where the profile will begin to be drawn.

Stop value specifies the point within the user-defined contour at which values drawing should stop.

The *IBufferID* and *PBufferID* specify the IDs of the two memory buffers that hold the array of incoming datastream (I) and output position (P) pairs which comprise the user-defined contour to be drawn.

1.2.2 Readable Parameters

The following table summarizes parameters associated with the user-defined profile mode that can be read by the host, but not written.

Profile Parameter	Format	Word Size	Range
Datastream Value	32.0	32 bits	-2,147,483,648 to 2,147,483,647
Datastream Index	32.0	32 bits	0 to 2,147,483,647
Contour Output	32.0	32 bits	-2,147,483,648 to 2,147,483,647
Contour Offset	32.0	32 bits	-2,147,483,648 to 2,147,483,647
Commanded Position	32.0	32 bits	-2,147,483,648 to 2,147,483,647

Below is a brief summary of these readable parameters. Each of these will be explained in more detail in the sections that follow.

Datastream Value and *Datastream Index* hold the instantaneous incoming datastream position or time and buffer index values of the active user-defined contour.

Contour output holds the instantaneous output position command of the active user-defined contour before being offset by the contour offset.

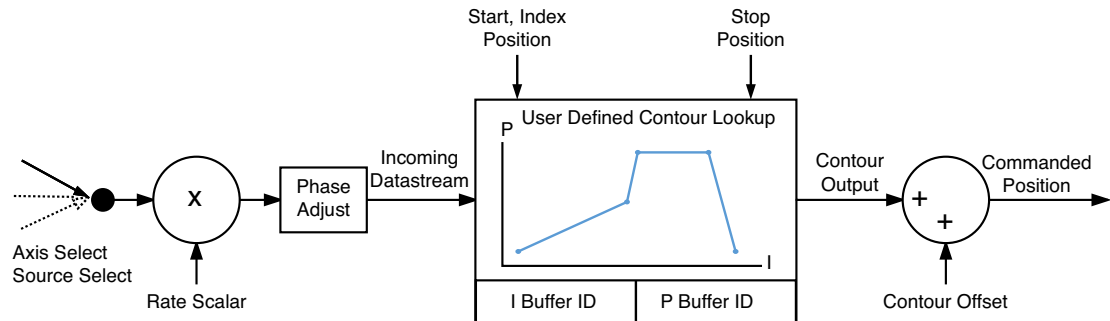
Contour offset holds an offset value that determines, with the contour output value, where the user-defined profile will be drawn within the overall position space of the driven axis.

Commanded position holds the instantaneous commanded output position of the driven axis which is in turn input to control elements such as the PID loop or pulse & direction generator. Note that this is the 'normal' commanded position register supported by all other Magellan profile modes.

1.3 User-Defined Profile Mode Operation

[Figure 1-3](#) provides a summary of the control flow of the user-defined profile mode.

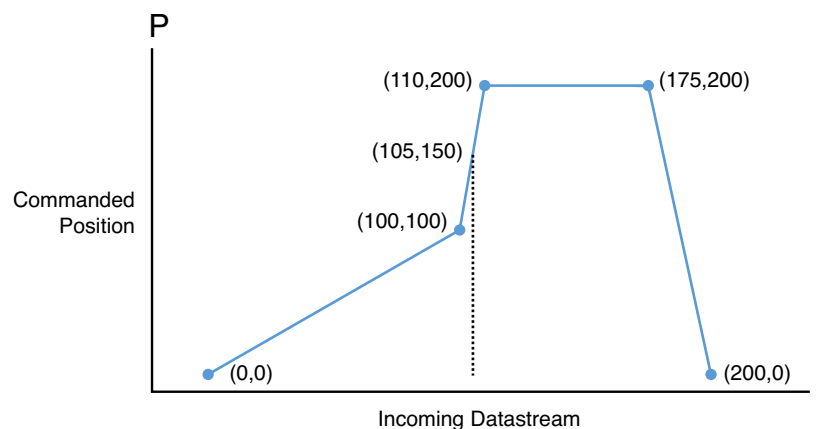
Figure 1-3:
Control Flow
Overview of
User-Defined
Profile Mode



The user-defined profile mode is a flexible system for utilizing a datastream of incoming positions (if the master source is set to actual position or commanded position) or time values (if the master source is set to time) to generate a corresponding continuous stream of commanded output positions. Because the relationship between the incoming datastream and the commanded output positions can be entirely arbitrary, any desired shape can be drawn using the user-defined profile mode.

[Figure 1-4](#) provides a simple example of a stored user-defined contour. The horizontal axis represents the incoming position or time datastream values, and the vertical axis represents the commanded output position of the driven axis. In this example the user-defined contour traverses 200 counts of the incoming datastream.

Figure 1-4:
Simple
Example User-
Defined
Contour



Five contour points are defined in the example, with each point being assigned a location within the incoming datastream denoted “I,” and a corresponding desired driven axis output position denoted “P.” This is summarized in the following table:

Point #	(I, P)
1	(0, 0)
2	(100, 100)
3	(110, 200)
4	(175, 200)
5	(200, 0)

The array of incoming datastream value (I) and commanded output position (P) pairs define how the input datastream will be used to calculate output commanded positions. For input values that exactly correspond to the user-defined profile points, the specified user-defined output values are output. In this example this means that if the incoming datastream value is, for example, 100 the output position is 100.

For input values that are in between user-defined profile points, linear interpolation is used to generate the position outputs. In this example this means that if the incoming datastream value is, for example, 105, the output position is 150.

There are some limitations to the specified values in I and P contour array. While specified I and P values in the contour array may be negative or positive, I values must always increase at each value entry throughout the entire array. In addition, while the total range of allowed I and P entries is -2,147,483,648 to +2,147,483,648, consecutive I value entries may not increase by more than 65,536.



1.3.1 User-Defined Profile Mode and Synchronization

The example user-defined profile in [Figure 1-4](#) describes a particular mapping between the master axis incoming data stream (which are positions or time values) to the driven axis commanded output positions.

This mapping approach to profile generation forms the core of the user-defined profile mode, and ensures that synchronization to the incoming datastream is automatic. If the mechanism or rate generator that provides the incoming datastream values slow down, traversal through the user-defined profile slows down in exact synchrony. Conversely, if the incoming datastream values speed up or even reverse direction, the user-defined profile will faithfully stay synchronized.

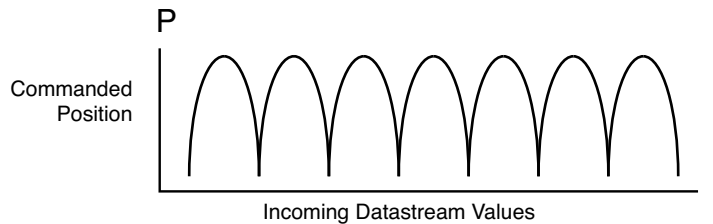
1.3.2 Master Axis Source Selection

The user-defined profile mode allows one of three master axis sources to be selected; actual position from an external encoder, commanded position from an internal profile, or time. The choice of the master axis source determines a number of important aspects of how the user-defined profile gets executed including the speed, whether the contour is synchronized to an external or internal source, and whether the contour is always traversed forwards, or may also be traversed backwards.

1.3.2.1 Actual Position as the Master Source

When the master axis source is set to actual position, an external encoder provides the incoming position datastream that is used to traverse the user-defined contour. As such, the rate and direction of how the user contour is traversed is entirely controlled by the encoder. The most common use of this type of master axis source is to execute a CAM function, where execution of the contour is synchronized to an external mechanism or other electronic signal source.

**Figure 1-5:
Example User-Defined
Contour in a
CAM
Application**

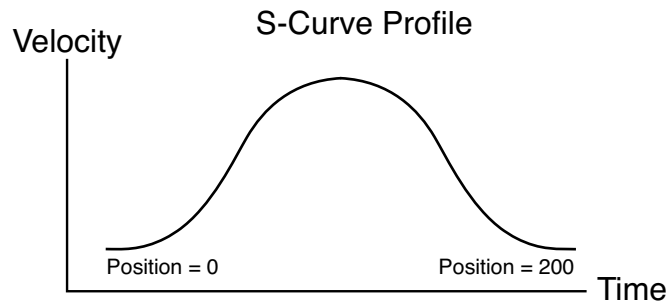


1.3.2.2 Commanded Position as the Master Source

When the master axis source is set to commanded position, the profile generator output of one of the Magellan's internal axes provides the incoming position datastream used to traverse the user-defined contour.

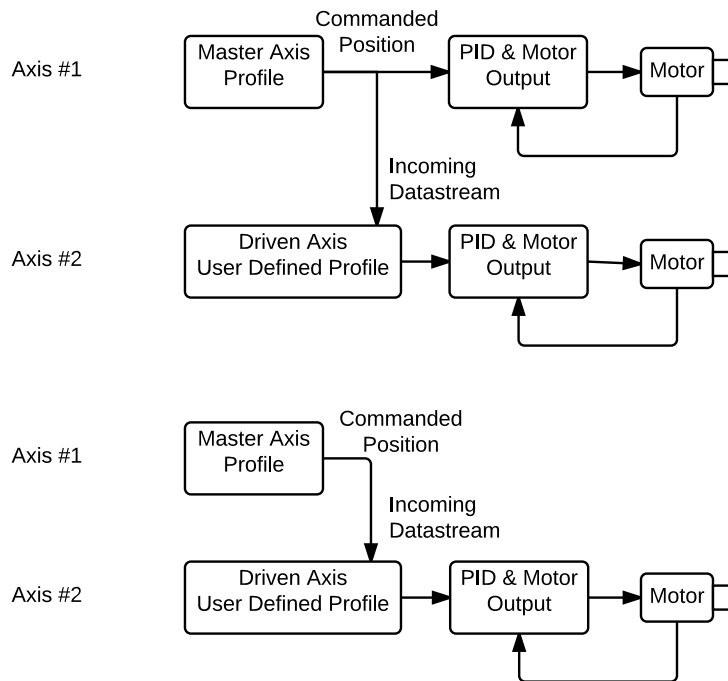
A common use of this approach is to allow velocity control of the user-defined profile. For example following the contour example of [Figure 1-4](#), if the master axis executes an S-curve profile from a position of 0 to a position of 200, the drawn user-defined profile will accelerate and decelerate to a smooth stop correspondingly. Such a master axis profile velocity contour is shown in [Figure 1-6](#).

**Figure 1-6:
Master Axis S-
Curve Profile**



Note that if the master axis instead executes a trapezoidal profile, then on-the-fly changes can be made allowing user-defined contour traversal to be slowed down, increased, reversed, or altered in any other way desired.

When used as the commanded position for a user-defined contour the master axis is not required to drive a motor directly. The master axis' sole purpose can be to provide a velocity envelope to one or more user-defined profiles. There are a number of applications however where it may be desirable for both the master axis and the driven axis to drive motors. These two alternate configurations of master and driven axes are shown in [Figure 1-7](#).



**Figure 1-7:
Master Axis
Motor
Connection
Schemes**

1.3.2.3 Time as the Master Source

When the master axis source is set to time, the Magellan's system clock provides the incoming datastream values that are used to traverse the user-defined contour. Unlike the other master axis sources, selecting time means that neither the rate nor the direction of user-defined profile traversal will change (although see [Section 1.3.3, Adjusting the Incoming Data Stream via the Rate Scalar](#)).

For those systems that can accommodate it, a key benefit of using time as the incoming datastream is that no dedicated internal axis is required for providing velocity control. Therefore all available axes can be used to drive output motors.

1.3.3 Adjusting the Incoming Data Stream via the Rate Scalar

As is shown in [Figure 1-3](#) a rate scalar register is available that can be used to re-scale the rate of the raw incoming datastream from the master. This register, which is 32 bits in size, defaults to a scale value of 1.0, meaning that the existing rate is passed through without rate change.

To reduce the datastream rate the user writes a value of less than 1.0, for example 0.5 (halving the rate). Conversely, to increase the rate a scalar of 2.0 (doubling the rate) can be used. To reverse the direction of the datastream a negative rate scalar may be specified.

To set the rate scalar register the command **SetProfileParameter** is used, which specifies a signed 32 bit number in 16.16 format. To retrieve the currently set scalar the command **GetProfileParameter** is used.

The rate scalar is most commonly used when the master source is set to time, to allow an approximation of smooth acceleration and deceleration. This is accomplished by sending a succession of rate scalars, each affecting a relatively small change in value, to increase or decrease the rate of traversal through the user-defined profile.

1.3.3.1 Rate Scalar and Abrupt Stop

A rate scalar with a selected value of 0.0 means that the user-defined profile is at a full stop. If desired, the user may manually set the rate scalar to this value to stop the profile.

The user may also specify an abrupt stop action, either directly or as part of a breakpoint or event action. Abrupt stop in the user-defined profile mode also uses the rate scalar, setting the rate scalar value to 0.0 to effect a stop of motion.

For more information on abrupt stops and breakpoint actions see the *Magellan Motion Control IC User's Guide*.



The User-defined Profile Mode does not directly support a smooth stop. Nevertheless, if a velocity envelope is used to command the incoming data stream value, (see [Section 1.3.2.2, Commanded Position as the Master Source](#) for more information) a smooth stop in the velocity envelope axis will result in a smooth stop of the user-defined contour.



Abrupt stops, once executed, result in the rate scalar being set to 0.0. After an abrupt stop, to restart motion the user must set the rate scalar to a non-zero value.

1.3.4 Setting a Stop Value

The section above described how the rate scalar can be used to explicitly stop the user-defined profile from executing. While useful, stopping the driven axis using this method is most often used in response to an event unrelated to the execution of the contour itself such as an emergency stop, or in response to some external signal or event.

It is also possible to explicitly set a stopping position for a given user-defined profile via the command **SetProfileParameter**. The user-defined profile stop value holds the incoming datastream value at which profile execution should stop. The specified stop value can be read back using the **GetProfileParameter** command.

Often, the stop value is equal to the I value of the last specified (I,P) user-defined profile point. In the example above from [Figure 1-4](#), assuming that normal traversal of the profile was in the positive direction, this would be a stop value of 200. Nevertheless, the stop value may be set to other values if, for example, traversal is in the negative direction, or if for whatever reason it is not desired that all of a particular loaded profile is executed.

Note that the stop value specifies a bidirectional traversal value, meaning that the stop condition will be satisfied whether traversed in a negative (decreasing values of the incoming value datastream) or positive direction (increasing values of the incoming datastream).

To render the stop value check inoperative, a stop value must be specified that can not be traversed by the incoming datastream. In the example from [Figure 1-4](#) this means any value less than 0 or larger than 200.

1.3.5 Adjusting the Phase of the Incoming Datastream

As can be seen in [Figure 1-3](#) a phase adjust register is provided that allows the raw incoming position or time datastream to be adjusted by the user. This may be used in a number of circumstances but is most often utilized to correct the phasing of a repeating cam so that the output user-defined profile is in-phase with an external mechanism or signal.

The phase adjust is specified in units of the incoming datastream values, and is applied immediately, and all at once. Positive phase adjust values move the incoming datastream value forward, while negative values move it backward.

After being applied the phase adjust register value returns to a value of zero. To set the phase adjust register the command **SetProfileParameter** is used.

The phase adjust register applies an instantaneous shift to the incoming datastream, and if applied during motion can therefore result in potentially abrupt changes of the driven axis' commanded output position. It is recommended that to affect larger shifts a series of smaller shifts be applied over a period of time, or that axis motion be stopped while the shift adjustment occurs. It is the responsibility of the user to insure that application of the phase adjust register does not result in undesired or dangerous motion.



1.3.6 User-Defined Contour Registers

There are a number of registers that hold user-defined contour-related quantities that may be useful for operations during contour drawing, or to restart a contour after it has been halted for any reason.

These registers include the datastream value, which holds the instantaneous value of the incoming datastream position or time value after rate scalar conversion and phase adjust, and the datastream index, which holds the corresponding instantaneous buffer index value of the incoming datastream value. To read either of these quantities the command **GetTraceValue** is used.

The instantaneous output of the user-defined contour lookup mechanism is stored in the contour output register. To read this quantity the command **GetTraceValue** is used.

1.4 Setting Up and Using User-Defined Contour Buffers

As described in previous sections, user-defined contours are represented as arrays of I, P pairs where I is the incoming datastream value, and P is the corresponding desired output position of the driven axis (the axis set to user-defined profile mode).

Each of the I and P arrays that comprise the user-defined contours are stored in separate buffers, utilizing the Magellan's standard buffer management commands. See the *Magellan Motion Control IC User's Guide* for background information on how to set up, load, and read from buffers.

There are no restrictions on the size of the I and P buffers, other than that they must be equal in size, and that the actual amount of accessible hardware buffer memory on the Magellan-based system or the internal memory of the MC58113 Series IC not be exceeded.

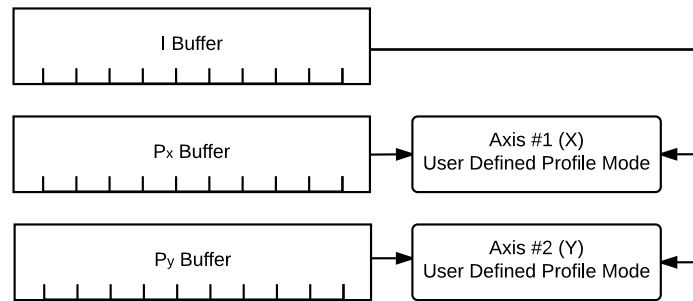
To set the BufferID for both the P and the I arrays the command **SetProfileParameter** is used. To read back these same buffer IDs the **GetProfileParameter** command is used.

1.4.1 Shared I Buffer Arrays and Multi-Axis Synchronization

There are no restrictions on multiple driven axes pointing to the same buffers. In particular, a common and very useful configuration is to have a single shared I array buffer with multiple corresponding P arrays to generate multi-dimensional synchronized moves.

For example, as shown in [Figure 1-8](#), two-dimensional (X, Y) synchronized path generation can be set up by specifying a single I array which has two rather than one associated P arrays. The Px array holds all the X output points and the Py array holds all the Y points. The X driven axis will specify the IBufferID and the PxBufferID, while the Y driven

Figure 1-8:
Example Two-Axis
Synchronization
Via Shared
IBuffer



axis will specify the same IBufferID and the PyBufferID. In addition, both the X driven axis and the Y driven axis must specify the same master axis number and source.

Once so initialized, traversal through both the X and Y user-defined profiles will be completely synchronized, allowing very precise generation of user-defined 2-D paths such as circles, arcs, lines, etc...

1.4.2 On-the-Fly Buffer Loading

In situations when the size of the user-defined profile to be drawn exceeds the size of the available memory buffer, an on-the-fly or rolling buffer approach may be used to continually feed in new (I, P) pairs to replace pairs already drawn.

For example, in the highly simplified example from the original user-defined object in [Figure 1-4](#) where the buffer size was 5 pairs long, if in fact the user-defined profile to be drawn is actually 7 pairs, a rolling buffer method could be used to first load in five pairs, and then load in the remaining two pairs once a sufficient portion of the first 5 pairs has been drawn and is no longer needed.

The primary challenge in loading new contour pairs on the fly is to ensure that only 'already drawn' contour pairs are overwritten, and that needed values for ongoing drawing are written in time. Both of these functions can be accomplished by utilizing the datastream index, which continually indicates the current index location of the user-defined profile as it is being executed. The datastream index is 32 bits, and can be read using the command **GetTraceValue**.

1.4.2.1 Automatic Reload Notification

In addition to directly reading the datastream index register, it is possible to set up a Magellan breakpoint that continually compares the datastream index against a user-specified index. Two breakpoints are available for this function, one that provides a greater or equal to comparison (for positive traversal direction of the contour), and one that provides a less than or equal to comparison (for negative direction traversal of the contour).

These breakpoints provide an efficient mechanism to alert the host that a data reload is needed rather than continually having the host check the progress of contour execution. For more information on setting up and processing breakpoints see the *Magellan Motion Control IC User's Guide*.

1.4.2.2 On-The-Fly Buffer Wrap

When loading buffer data that will be executed after a wrap from the last buffer entry to the first, the first (I, P) pair must be an exact copy of the last. From the example in [Figure 1-4](#), when traversing the user-defined contour in the

positive direction, this means that the portion of the contour to be loaded on-the-fly must first load in a copy of the pair at Index 5 into Index 1. The following tables shows this:

Buffer Index #	Stored (P, I) values
1	(0, 0)
2	(100, 100)
3	(110, 200)
4	(175, 200)
5	(200, 0)
1	(200, 0)
2	(300, 125)
3	(400, 75)

In the above example two new user-defined points are being added; (300, 125) and (400, 75). They are loaded at index #2 and #3 respectively, because a copy of the last pair is required in index #1.

Note that in the above example a compare breakpoint of 3, representing buffer index 3 may have been used to trigger the on-the-fly reload, and if it is desired that the profile automatically stop after executing the last user-defined profile point, a stop value of 400 should be programmed.

1.4.3 Buffer Wrap Without On-the-Fly Data Loading

In the above section buffer wrap in the context of loading more user-defined profile data was described. It is also possible however for the positively or negatively traversing incoming datastream values to result in a buffer wrap without loading new contour data.

In this scenario the wrap still occurs, however the incoming datastream value undergoes a modulo shift to re-reference it to the beginning of the buffer so that the datastream value agrees with the already-loaded user-defined (I, P) contour points. From the example in [Figure 1-4](#), this means that as the incoming datastream value exceeds 200, it is re-referenced by subtracting 200, so that 201 becomes 1, 202 becomes 2, etc. and the contour will continue to execute.

The effect of this is that the same contour will be executed continuously as long as the incoming datastream continues to positive or negatively traverse the contour, thereby resulting in ongoing buffer wraps. This is shown in [Figure 1-9](#).

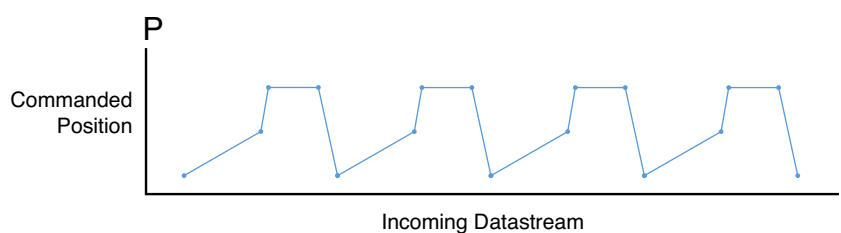


Figure 1-9:
Example
Position Output
With Zero
Displacement
Wrap

There is no limit to how many times the user-defined contour can be re-traversed in this manner, whether in the positive or negative direction. Used in this way the continuous contour profile mode acts as a repeating cam, where a continually repeating pattern is driven from a linearly increasing position.

1.4.3.1 Stop Value With Automatic Wrap

In this 'auto-wrap' mode, the stop value function, to be effectively used, must refer to a position within the re-referencing range. In the above example this means the stop value must be within the range of 0 to 200, and thus the stop value function can not be used to stop after (for example) a specific number of traversals of the cam profile.

If such a functionality is desired the stop position should be set outside of the re-referencing range (effectively turning it off) during the first two cam traversals, and then set to the desired stop position within the re-referencing range during the third traversal. For more information on the stop position function see [Section 1.3.4, Setting a Stop Value](#).

1.4.4 Buffer Wrap With Position Displacement

In the example of [Figure 1-4](#), the first (I, P) pair and the last (I, P) pair both command the same driven axis position of 0. If, however, the pre and post-wrap positions are not the same, the commanded position will automatically be displaced by the difference in the two positions.

To illustrate this, assume the following user-defined profile is specified, which is identical to the example contour except that the last point finishes at 50 rather than 0:

Buffer Index #	Stored (P, I) values
1	(0, 0)
2	(100, 100)
3	(110, 200)
4	(175, 200)
5	(200, 50)

When executing this user-defined contour and when wrapping from the last contour pair to the first, the commanded output position for point 1 and beyond is incremented by $P_5 - P_1$, or by 50 in this example. As long as the contour data is not changed on-the-fly, each time a wrap occurs, the output position shifts by +50. This is shown in [Figure 1-10](#).

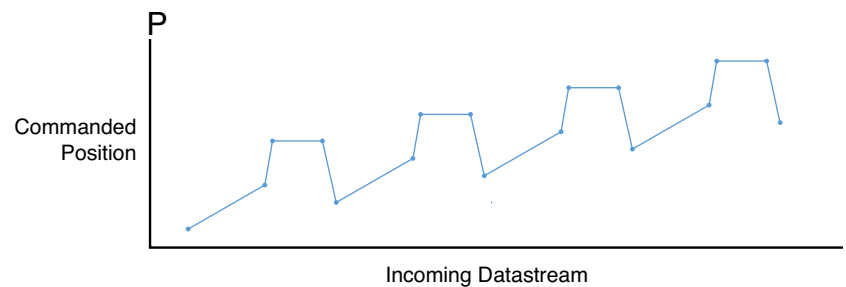


Figure 1-10:
Example
Position Output
With Non-Zero
Displacement
Wrap

Note that if this same user-defined profile is traversed in the negative direction, the shift is -50 at each wrap, and the net displacement of the user-defined profile is in the negative rather than positive direction.

1.5 Controlling Contour Drawing

User-defined contours specify how a particular contour varies in relation to itself. The stored contour does not however specify important information such as from where within the contour drawing should start, and to where, within the addressable space of the driven axis, it should be drawn.

1.5.1 Contour Drawing Start Location

To specify from where within the contour drawing begins both an incoming datastream start index and datastream start value must be specified. For example using the contour from [Figure 1-4](#), if it is desired that drawing starts at the (110, 200) I, P point a start index of 3 is specified, and a start value of 110 is specified.

It is also possible to start drawing at interpolated incoming datastream values. Again, from the the contour in [Figure 1-4](#), a start value of, for example, 142 can be specified which corresponds to a start index of 3. In this case the

output will automatically be interpolated at the incoming datastream value at 142 so that the correct output value is output from the start of motion onward.

To set the start index and start value the command **SetProfileParameter** is used. To read back these values the **GetProfileParameter** command is used.

1.5.2 Contour Drawing Output Position

To reference the drawing location of a stored contour within the addressable space of the output driven axis one of the Magellan's direct position command profiles, such as Trapezoidal Profile, is used to move the axis to the desired start location. Once at the correct location the axis is switched to user-defined profile mode and contour drawing can begin.

An example of this is shown in [Figure 1-11](#), where two driven axes have been set up with appropriate user-defined profiles to draw a lower case “a,” and where this object is drawn three separate times beginning at three separate locations.

In each case a direct position profile mode such as trapezoidal or S curve point-to-point is used to locate the driven axes to the correct start locations. Then the user-defined profile parameters are set up and the profile is switched to user-defined to draw each “a” contour.

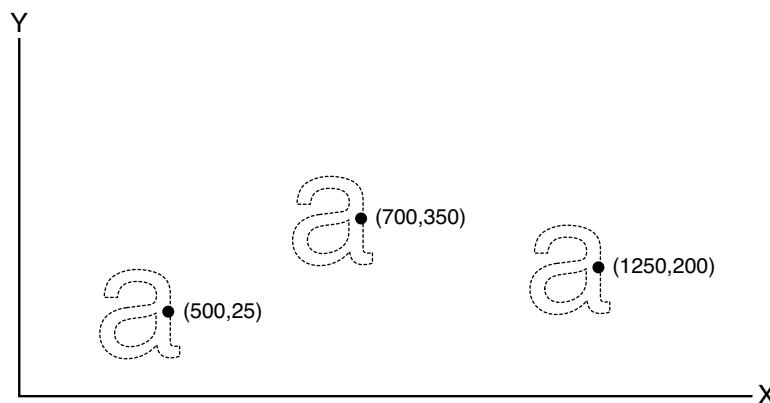


Figure 1-11:
Stored Two-Dimensional Contour Drawn in Three Separate Locations

The mechanism for managing the object drawing location is the contour offset register, shown in [Figure 1-3](#). This register can be read, but cannot be directly written to. It is set automatically upon entering the user-defined profile mode, or when the rate scalar changes from a zero value to a non-zero value.

To read the contour offset register the command **GetTraceValue** is used.

1.6 Entering and Exiting User-Defined Profile Mode

Caution should be exercised when entering the user defined profile mode, since depending on how the various profile control parameters described in [Section 1.2.1, Settable Parameters](#), have been set up, the profile could start drawing immediately.

The safest approach is to load all the user-defined profile parameters before entering the user-defined profile mode. Alternatively, the rate scalar described in [Section 1.3.3, Adjusting the Incoming Data Stream via the Rate Scalar](#), can

be set to 0.0 before entering user defined profile mode, and then other parameters can be set, followed by a start of profile drawing via setting of the rate scalar to 1.0 or some other non-zero value.

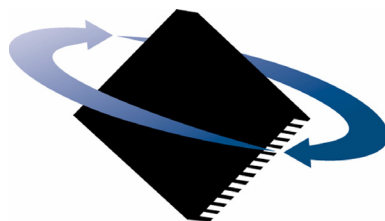


It is the responsibility of the user to ensure that stored contours and other control settings for the user-defined profile mode are safe for the intended mechanisms to be controlled.

For additional information, or for technical assistance,
please contact PMD at (978) 266-1210.

You may also e-mail your request to support@pmdcorp.com

Visit our website at <http://www.pmdcorp.com>



P M D

Performance Motion Devices, Inc.
80 Central Street
Boxborough, MA 01719

User-Defined Profile Addendum

The Magellan Motion Control IC User-Defined Profile Mode Documentation describes the features and use of the user-defined profile mode, but not the low level command interface. This document specifies the opcodes and other command details required to program a user-defined profile application.

Set/GetProfileMode

The standard Set/GetProfileMode commands are described in the Magellan Programmer's Command Reference. For Magellan versions supporting the user-defined profile mode an additional instance of the mode argument is available:

The encoding 10 (ten) specifies the user defined mode.

Set/GetTraceVariable

Several custom trace variables are provided as part of the user-defined profile custom. These variable indices may be used as arguments to SetTraceVariable, GetTraceVariable, and GetTraceValue.

Instance	Encoding	Range/Scaling
DataStream value	90	32 bit signed
DataStream Index	91	32 bit unsigned
Contour Output	92	32 bit signed
Contour Offset	93	32 bit signed
Active Rate Scalar	94	signed, 1/65536

GetEventStatus

A new event bit is defined as part of the user-defined profile custom. Bit 13 (mask 2000h) indicates that a runtime error has been raised. For more information see the description of GetRuntimeError below. This bit is meaningful when getting or clearing event status, setting event-driven breakpoints, and setting the fault out mask.

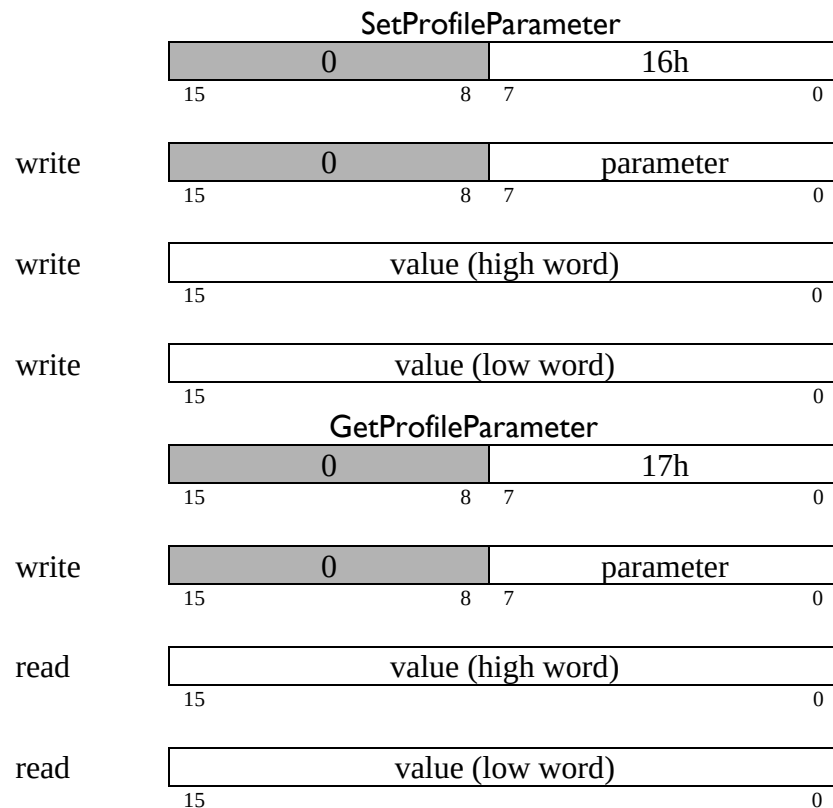
Set/GetProfileParameter

Syntax SetProfileParameter axis parameter value
 GetProfileParameter axis parameter

Arguments

Name	Instance	Encoding
parameter	Source	3
	Rate Scalar	7
	Start Value	9
	Start Index	8
	Stop Value	10
	PBufferId	0
	IBufferId	6
value	Type	Range/Scaling
	Signed or unsigned 32 bit	See below

Packet Structure



Description **Set/GetProfileParameter** are used to store and retrieve various parameters controlling the behavior of the user-defined profile mode. All parameter values not described are reserved, possibly for other profile modes. All values are read and written as 32 bit quantities but range and whether signed or unsigned depends on the parameter.

Except as noted below this command is unbuffered, that is, changes take effect immediately instead of after an **update** command.

The **Source** parameter is a bitmask, bits [7:0] specify the source axis, 0 means Axis1, 1 means Axis2, and so forth. Bits [15:8] specify the source type, 0 means actual position, 1 means commanded position, and 2 means time (axis field ignored). All other values are reserved.

The **Rate Scalar** is a signed quantity scaled by 1/65536, that is, a 16.16 fixed point quantity. The **Rate Scalar** is buffered.

The **Start Value** and **Stop Value** are signed 32 bit integers. The **Stop Value** is buffered.

The **Start Index** is an unsigned 32 bit integer.

The **PBufferId** and **IBufferId** are unsigned 5 bit integers specifying memory buffers, so they have a maximum value of 31.

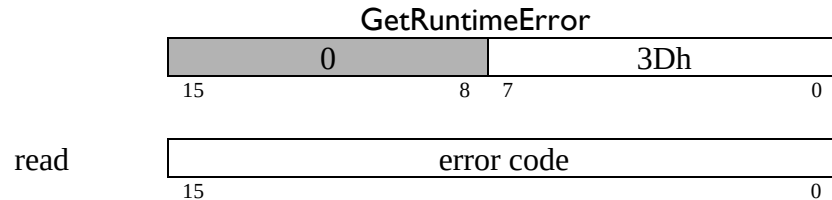
C-Motion API

```
PMDresult PMDSetProfileParameter(PMDAxisInterface axis_intf,  
                                  PMDuint16 parameter,  
                                  PMDuint32 value);  
  
PMDresult PMDGetProfileParameter(PMDAxisInterface axis_intf,  
                                  PMDuint16 parameter,  
                                  PMDuint32* value);
```

GetRuntimeError

Syntax GetRuntimeError axis

Packet Structure



Description The **GetRuntimeError** command is used to retrieve a 16-bit code indicating a runtime error, that is, an error not directly associated with a specific command. When a run-time error occurs the RuntimeError bit in the event status register will be set, and, if the internal runtime error register is zero then it will be set to a nonzero code indicating the error.

Reading the error code sets the internal runtime error register to zero, but does not clear the RuntimeError event bit, which must be done separately using the **ResetEventStatus** command.

The currently defined runtime error codes, shown below, are all associated with the user-defined profile. Any codes not described here are reserved. Future error codes may be added to indicate problems with other standard or custom facilities.

Instance	Encoding	Notes
Error in I table	1	Entries are not strictly increasing, or length is less than 2, or otherwise unusable.
Table overrun	2	The source value changed too quickly, moving over more than one table entry.
Error in P table	3	Length is not the same as I table, or otherwise unusable.
Index error	4	Starting index not in table range.

C-Motion API

```
PMDresult PMDGetRuntimeError(PMDAxisInterface axis_intf,
                               PMDuint16* error);
```

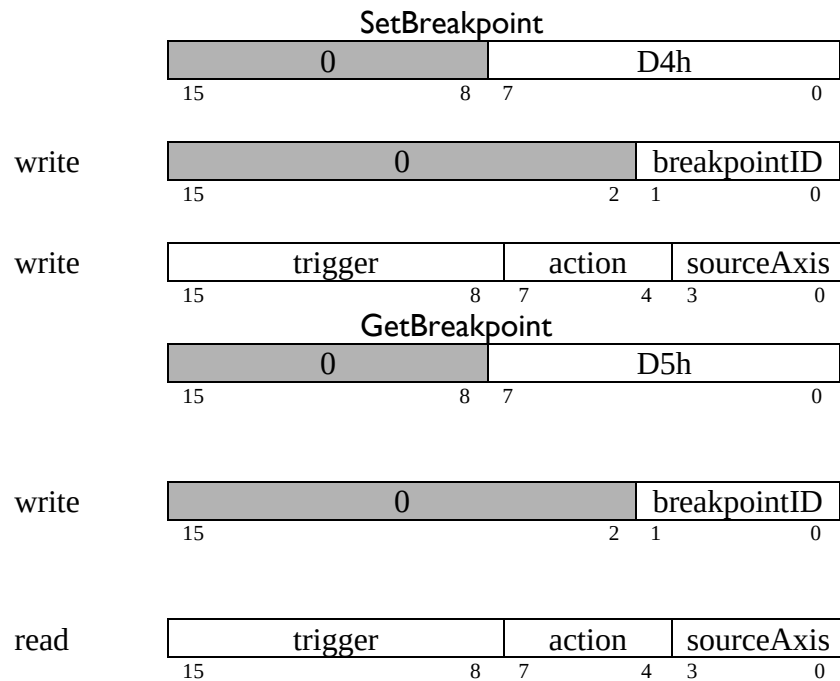
Set/GetBreakpoint

Syntax SetBreakpoint axis breakpointID sourceAxis action trigger
 GetBreakpoint axis breakpointID

Arguments

Name	Instance	Encoding
trigger	Table index >= value	14
	Table index <= value	15

Packet Structure



Description Set/GetBreakpoint are standard commands that have been extended by two possible breakpoint triggers.

A breakpoint with trigger 14 will occur when the user defined profile table index becomes greater than or equal to the specified breakpoint value. A breakpoint with trigger 15 will occur when the user defined profile table index becomes less than or equal to the specified breakpoint value. As usual, the breakpoint value should be set before setting the breakpoint.

The difference between the breakpoint value and the current index is evaluated as the shortest distance allowing for buffer wrap.

For example, suppose the buffer length is 100, the breakpoint value is 10 and the greater than or equal trigger is set. If the current index value is 61 then the breakpoint will not fire, because the distance in the positive direction is $61 - 10 = 51$, while the distance in the negative direction is $(10 - 61) \bmod 100 = 49$.

These triggers may be useful if the user-defined buffers must be written while the profile is active. If the behavior of the master source is predictable it is possible to trigger a breakpoint when it is safe to write to a specified part of the buffers.

The behavior of breakpoints with these triggers when user-defined profile mode is not active is not defined. It is not safe to set a table index breakpoint before the update that starts user-defined profile mode.

C-Motion API

```
PMDresult PMDSetBreakpoint(PMDAxisInterface axis_intf,  
                           PMDuint16 breakpointID,  
                           PMDAxis SourceAxis,  
                           PMDuint8 action,  
                           PMDuint8 trigger);  
PMDresult PMDGetBreakpoint(PMDAxisInterface axis_intf,  
                           PMDuint16 breakpointID,  
                           PMDAxis *SourceAxis,  
                           PMDuint8 *action,  
                           PMDuint8 *trigger);
```